

ІНСТРУКЦІЯ ДО ПРАКТИЧНОЇ РОБОТИ №21

Тема: Створення алгоритмів на цикли

Дисципліна: Алгоритмізація і методи обчислень

Спеціальність: Професійна освіта (Цифрові технології)

Тривалість: 80 хвилин

Мета роботи

Опанувати структуру «цикл»: цикли з передумовою (while) та з параметром (for), вкладені цикли, оператори break і continue; навчитися застосовувати лічильники й накопичувачі, трасувати цикли та обґрунтовувати їхню скінченність і коректність.

Матеріально-технічне забезпечення

Персональний комп'ютер, середовище програмування (Python/C++/Pascal).

Короткі теоретичні відомості

1. Цикл — структура, що забезпечує багаторазове виконання групи команд (**тіла циклу**). Одне виконання тіла називають **ітерацією**. Кожен цикл має: початкові значення змінних, умову продовження (або діапазон), тіло та **зміну змінної, що веде до завершення**.

2. Цикл із передумовою `while` виконується, доки умова істинна; умова перевіряється **перед** кожною ітерацією, тому тіло може не виконатись жодного разу. Використовується, коли кількість ітерацій заздалегідь невідома (виділення цифр, пошук, збіжність).

```
while умова:
    тіло

for змінна in range(початок, кінець, крок):
    тіло
```

3. Цикл `for` із `range`. Вираз `range(a, b, h)` породжує значення `a`, `a+h`, `a+2h`, ... **строго менші за `b`** (права межа не включається): `range(1, 6)` дає 1, 2, 3, 4, 5; `range(10, 0, -3)` — 10, 7, 4, 1. Цикл `for` застосовують, коли кількість ітерацій відома.

4. Типові прийоми:

- **лічильник** — змінна, що рахує кількість подій (`count += 1`);
- **накопичувач суми** — починається з 0 (`s += x`); **накопичувач добутку** — починається з 1 (`p *= x`);
- **прапорець** — логічна змінна, що фіксує, чи сталася подія;
- **пошук максимуму/мінімуму** – початкове значення беруть з першого елемента, а не «0» чи «-1»;
- **сторожове значення (sentinel)** — спеціальне значення, що завершує введення (наприклад, 0).

5. Оператори керування циклом: `break` — негайно завершує цикл; `continue` — переходить до наступної ітерації; блок `else` після циклу виконується, якщо цикл завершився **без** `break`.

6. Вкладені цикли. Для кожної ітерації зовнішнього циклу внутрішній виконується повністю. Якщо зовнішній має `n` ітерацій, а внутрішній — `m`, тіло внутрішнього виконається `n*m` разів.

Якщо межа внутрішнього залежить від лічильника зовнішнього (`for j in range(i)`), кількість виконань — $0 + 1 + \dots + (n-1) = n(n-1)/2$.

7. Коректність і скінченність. Інваріант циклу — твердження, яке істинне перед і після кожної ітерації (наприклад, для суми: «s дорівнює сумі вже оброблених елементів»). Скінченність забезпечується тим, що змінна в умові **монотонно** наближається до завершення. Типові помилки: нескінченний цикл (забули змінити змінну), помилка «на одиницю» (off-by-one) у межах, неправильне початкове значення накопичувача.

Хід роботи

Завдання 1. Трасування циклу while: послідовність Коллатца

Для натурального n виконують: якщо n парне — ділять його на 2, інакше замінюють на $3n + 1$; процес повторюють, доки n не стане рівним 1. Виконати вручну трасування цього алгоритму.

```
алг Коллатц
  ввести n;  steps := 0
  поки n ≠ 1
    якщо n парне то n := n div 2 інакше n := 3 * n + 1
    steps := steps + 1
  все
  вивести steps
кін
```

Порядок виконання:

1. Побудуйте таблицю трасування зі стовпцями «ітерація», « n до кроку», «парне?», « n після кроку», «steps».
2. Виконайте алгоритм для $n = 6$, записуючи всю послідовність значень n до досягнення 1.
3. Виконайте алгоритм для $n = 7$ та порівняйте кількість кроків із результатом для $n = 6$.
4. Запишіть, для яких значень n тіло циклу не виконається жодного разу, і чому.
5. Поясніть, чому для цього алгоритму **невідомо**, чи завершується він для будь-якого n (гіпотеза Коллатца), і яку умову скінченності потрібно було б довести.

Завдання 2. Трасування вкладених циклів

Виконати вручну фрагмент програми з вкладеними циклами, визначити його вивід і кількість ітерацій.

```
for i in range(1, 5):
    for j in range(1, i + 1):
        print(i * j, end=" ")
    print()
```

Порядок виконання:

1. Запишіть значення i , для кожного i — діапазон значень j , а також усі вирази $i \cdot j$, що виводяться.
2. Побудуйте таблицю « i — значення j — виведений рядок» і запишіть повний вивід програми.
3. Порахуйте, скільки разів виконається тіло внутрішнього циклу (кількість викликів `print` із виразом $i \cdot j$) для кожного i і загалом; порівняйте з формулою $n(n+1)/2$ при $n = 4$.

4. Змініть внутрішній цикл на `range(1, 5)` і визначте нову кількість ітерацій; поясніть різницю.
5. Складіть трасування для фрагмента `s = 0 / for i in range(1, 6): / if i % 2 == 0: continue / s += i` і з'ясуйте значення `s`; поясніть роль `continue`.

Завдання 3. Програмна реалізація циклів: лічильники й накопичувачі

Реалізувати мовою Python типові циклічні алгоритми та перевірити їх тестами й ручними розрахунками.

Порядок виконання:

1. Напишіть функцію `factorial(n)` на циклі `for` (накопичувач добутку з початковим значенням 1).
2. Напишіть функцію `digit_stats(n)`, що за допомогою циклу `while` виділяє цифри натурального числа та повертає кількість цифр, їх суму та найбільшу цифру.
3. Напишіть функцію `average_until_zero(values)`, що імітує введення чисел до сторожового значення 0 (цикл `while` по індексу) і повертає кількість введених чисел та їх середнє арифметичне.
4. Напишіть функцію `first_power_above(limit)`, що знаходить найменший степінь двійки, який перевищує задане число, і повертає показник та значення.
5. Виведіть за допомогою вкладених циклів таблицю множення 1...9 у форматованому вигляді (перші 5 рядків).
6. Перевірте функції на 2–3 тестах кожен, у тому числі на граничних значеннях (`n = 0`, `n = 1`, порожній ввід).

Приклад реалізації мовою Python:

python

```
def factorial(n):
    p = 1
    for k in range(2, n + 1):
        p *= k
    return p

def digit_stats(n):
    count, total, biggest = 0, 0, 0
    while n > 0:
        digit = n % 10
        count += 1
        total += digit
        if digit > biggest:
            biggest = digit
        n //= 10
    return count, total, biggest

def average_until_zero(values):
    i, count, total = 0, 0, 0
    while i < len(values) and values[i] != 0:
        total += values[i]
        count += 1
        i += 1
    return (count, total / count) if count else (0, None)

def first_power_above(limit):
    k, value = 0, 1
    while value <= limit:
        value *= 2
        k += 1
    return k, value
```

```

print("0! =", factorial(0), "; 5! =", factorial(5), "; 20! =", factorial(20))
print("digit_stats(90417) =", digit_stats(90417))
print("digit_stats(7) =", digit_stats(7))
print("average_until_zero([4, 8, 6, 0, 9]) =", average_until_zero([4, 8, 6, 0, 9]))
print("average_until_zero([0, 5]) =", average_until_zero([0, 5]))
print("first_power_above(1000) =", first_power_above(1000))
print("first_power_above(10**6) =", first_power_above(10**6))

print("\nТаблиця множення:")
for i in range(1, 6):
    row = ""
    for j in range(1, 10):
        row += f"{i * j:4d}"
    print(row)

```

Результат виконання програми:

```

0! = 1 ; 5! = 120 ; 20! = 2432902008176640000
digit_stats(90417) = (5, 21, 9)
digit_stats(7) = (1, 7, 7)
average_until_zero([4, 8, 6, 0, 9]) = (3, 6.0)
average_until_zero([0, 5]) = (0, None)
first_power_above(1000) = (10, 1024)
first_power_above(10**6) = (20, 1048576)

```

Таблиця множення:

1	2	3	4	5	6	7	8	9
2	4	6	8	10	12	14	16	18
3	6	9	12	15	18	21	24	27
4	8	12	16	20	24	28	32	36
5	10	15	20	25	30	35	40	45

Висновок за результатами: усі функції дають очікуваний результат, у тому числі на граничних значеннях: $0! = 1$ (цикл не виконується, і накопичувач добутку залишається рівним початковій 1), для порожнього вводу середнє не обчислюється. Найменший степінь двійки, що перевищує 1000, — $2^{10} = 1024$. Початкове значення накопичувача добутку 1 (а не 0) принципово: інакше весь добуток був би нульовим. Таблиця множення ілюструє роботу вкладених циклів: зовнішній задає рядок, внутрішній — стовпці.

Завдання 4. Дослідження послідовності Коллатца та циклів із break / else

Реалізувати алгоритм Коллатца, знайти початкове число з найдовшою траєкторією в заданому діапазоні та дослідити кількість ітерацій вкладених циклів.

Порядок виконання:

1. Напишіть функцію `collatz_steps(n)`, що повертає кількість кроків до досягнення 1 (алгоритм із завдання 1); перевірте її на $n = 6$ і $n = 7$ (результати мають збігатися з ручним трасуванням).
2. Напишіть функцію `longest_collatz(limit)`, що для всіх n від 1 до `limit` знаходить число з найбільшою кількістю кроків і повертає пару (n , кроки).
3. Виконайте пошук для `limit = 10, 100, 1000, 10000` і занесіть результати в таблицю «межа — число — кількість кроків».
4. Напишіть функцію `first_divisible(start, a, b)`, що за допомогою `for ... else` знаходить перше число з діапазону `[start, start + 100)`, яке ділиться одночасно на a і b ; якщо такого немає — повертає `None` (обробка гілки `else`).
5. За допомогою лічильника порахуйте кількість виконань тіла внутрішнього циклу у вкладеній конструкції `for i in range(n): for j in range(i): ...` для $n = 10, 100, 1000$ і перевірте формулу $n(n-1)/2$.

Приклад реалізації мовою Python:

python

```
def collatz_steps(n):
    steps = 0
    while n != 1:
        if n % 2 == 0:
            n = n // 2
        else:
            n = 3 * n + 1
        steps += 1
    return steps

def longest_collatz(limit):
    best_n, best_steps = 1, 0
    for n in range(1, limit + 1):
        s = collatz_steps(n)
        if s > best_steps:
            best_n, best_steps = n, s
    return best_n, best_steps

def first_divisible(start, a, b):
    for x in range(start, start + 100):
        if x % a == 0 and x % b == 0:
            return x
    else:
        return None

print("collatz_steps(6) =", collatz_steps(6), "; collatz_steps(7) =", collatz_steps(7))
print("Межа    число    кроків")
for limit in (10, 100, 1000, 10000):
    n, s = longest_collatz(limit)
    print(f"{limit:>5}    {n:>6}    {s:>6}")

print("first_divisible(1, 6, 15) =", first_divisible(1, 6, 15))
print("first_divisible(1, 40, 77) =", first_divisible(1, 40, 77))

print("\n n    ітерацій внутрішнього циклу    n(n-1)/2")
for n in (10, 100, 1000):
    counter = 0
    for i in range(n):
        for j in range(i):
            counter += 1
    print(f"{n:>4}    {counter:>26}    {n * (n - 1) // 2:>8}")
```

Результат виконання програми:

```
collatz_steps(6) = 8 ; collatz_steps(7) = 16
Межа    число    кроків
  10         9        19
 100        97       118
 1000       871      178
10000      6171     261
first_divisible(1, 6, 15) = 30
first_divisible(1, 40, 77) = None

 n    ітерацій внутрішнього циклу    n(n-1)/2
 10                45                45
 100               4950             4950
1000              499500           499500
```

Висновок за результатами: для $n = 6$ алгоритм робить 8 кроків, для $n = 7 - 16$, що збігається з ручним трасуванням. Траєкторія числа не зростає монотонно

(наприклад, $7 \rightarrow 22 \rightarrow 11 \rightarrow 34 \rightarrow \dots$), тому кількість кроків неможливо передбачити за розміром числа: у діапазоні до 10000 найдовший шлях має число 6171 (261 кроків). Кількість ітерацій вкладеного циклу з межею `range(i)` точно дорівнює $n(n-1)/2$ – зростає квадратично: збільшення n у 10 разів збільшує кількість ітерацій приблизно у 100 разів. Перше число, що ділиться на 6 і 15 одночасно, – 30 (НСК), а для пари (40, 77) у діапазоні $[1, 101)$ числа немає, тому спрацьовує гілка `else i повертається None`.

Контрольні питання

1. Що таке цикл, ітерація, тіло циклу? Які складові повинен мати цикл?
2. Чим відрізняються цикли `while` і `for`? Коли доцільно використовувати кожен із них?
3. Які значення породжує `range(2, 11, 3)` і чому права межа не входить у діапазон?
4. Що таке лічильник, накопичувач суми та накопичувач добутку? З яких значень їх ініціалізують?
5. Як працюють оператори `break` і `continue`? Коли виконується блок `else` після циклу?
6. Скільки разів виконається тіло внутрішнього циклу в конструкції `for i in range(n) : for j in range(i) : ?`
7. Що таке інваріант циклу і як він допомагає довести коректність алгоритму?
8. Які причини виникнення нескінченного циклу? Як гарантувати скінченність циклу `while`?

Зміст звіту

1. Тема, мета роботи.
2. Таблиці трасування циклів (завдання 1, 2).
3. Тексти програм та результати їх роботи (завдання 3, 4).
4. Таблиця результатів пошуку найдовшої траєкторії Коллатца.
5. Висновки.